EL-2003 – Computer Organization & Assembly Language Lab
Lab # 6

Objectives:

1. Shift and rotate numbers wider than one register (32-bit) by chaining shl/rcl and shr/rcr through the carry flag.
2. Add multi-word numbers (64-bit) with add followed by adc, so the carry travels between words.
3. Multiply numbers larger than 16 bits by splitting them into words, using mul on each part and combining the partial products with add/adc.
4. Use the bitwise instructions and, or, xor, not and test to isolate, set, clear, toggle and check individual bits.
5. Combine masks and shifts to unpack values stored in packed bit-fields.
6. Verify results in AFD by reading registers, flags and the memory dump.

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

Question 1:

Declare a 32-bit number in memory as two words, low word first: num: dw 0x91E4, 0x2B6D (the number is 2B6D91E4h). Declare two more 32-bit variables, left4 and right3, each as two words initialised to 0. First, copy num into DX:AX (DX = high word, AX = low word) and shift this 32-bit value left by 4 positions using a loop that runs 4 times and executes shl ax,1 followed by rcl dx,1 on each pass. Store the result in left4 (low word first). The expected result is B6D91E40h. Next, copy num into DX:AX again and shift it right by 3 positions, this time using shr dx,1 followed by rcr ax,1 inside a loop that runs 3 times. Store the result in right3. The expected result is 056DB23Ch.

Question 2:

Part (a): Declare two 64-bit numbers as arrays of four words each, low word first: a: dw 0xFFFF, 0xFFFF, 0x1234, 0x0000 and b: dw 0x0001, 0x0000, 0x0001, 0x0000. Declare sum as four words initialised to 0. Add the two numbers word by word, using add for the lowest word and adc for the three words above it, and store each result word in sum. The expected sum is 0000, 0000, 1236, 0000.

Part (b): A 32-bit number is stored as big: dw 0x86A0, 0x0001 (it equals 100000), and the multiplier is the 16-bit value 25. Multiply the number by 25 without using any 32-bit instruction. Multiply the low word by 25 with mul and keep the result in DX:AX. Save AX as the low word of the answer and DX as a carry-over. Then multiply the high word by 25, add the carry-over, and store this as the high word of the

answer. Save the final 32-bit product in prod (two words, low word first).

Question 3:

Declare a byte flags: db 0x5A and reserve one byte each for res_or, res_and, res_xor, res_not, res_t4 and res_t5. Starting from a fresh copy of flags in AL each time, set bits 0 and 7 using or and store the result in res_or (expected DBh). Next, clear bits 1 and 3 using and and store the result in res_and (expected 50h). Toggle bits 2 and 6 using xor and store the result in res_xor (expected 1Eh). Invert all bits using not and store the result in res_not (expected A5h). Then use test to check whether bit 4 of flags is set, and store 1 or 0 in res_t4 (expected 1). Do the same for bit 5 and store the result in res_t5 (expected 0). Finally, load AL = 3Ch and BL = C3h and swap them using only three xor instructions, with no temporary register, so that AL ends as C3h and BL ends as 3Ch.

Question 4:

The registrar's office stores each date in a single 16-bit word to save space. The day sits in the lowest 5 bits, the month in the next 4 bits, and the year, counted from 1980, in the top 7 bits. Three dates are saved in an array: dates: dw 0x5B4B, 0x2E67, 0x279F. The office wants a small program that takes this array and produces three separate arrays holding the day, month and full year (for example 2025, not 45) of every record. After unpacking, the program should also work out how many of the dates fall in the last quarter of the year (October to December) and which record has the earliest year, and store both results in variables. Decide how to isolate each field and how to organize the arrays and variables, then write the program.

Question 5:

A teacher wants to show students how fast factorials grow, but the 8086 mul instruction only multiplies 16-bit values and gives at most a 32-bit result. A 16-bit register runs out of room at 8!. Write a program that computes 12! and keeps the answer in a 32-bit variable, building it step by step as $1 \times 2 \times 3 \times \ldots \times 12$ by reusing the multiplication idea from Question 2. Once you have the answer, the teacher also wants to know how many bits in the 32-bit result are set to 1, so count them with a loop of your own design that uses extended shifting and store the count in a variable. Decide your own variable names and layout.